

Saving data to MongoDB

Debugging Node applications

- Ways to Debug Node.js Apps:

- `console.log()` (The #1 Tool used by 95% of Engineers):

```
console.log('Incoming Body:', request.body);  
console.log('Found Person:', person);
```

- Chrome DevTools for Node (**`node --inspect index.js`**):

- You can run Node with `--inspect` and open `chrome://inspect` in your browser to debug Node backend code using Chrome's DevTools console!

- When bugs occur, the worst of all possible strategies is to continue writing code. It will guarantee that your code will soon have even more bugs, and debugging them will be even more difficult

MongoDB

- The reason for using Mongo as the database is its lower complexity compared to a relational database.
 - Document databases differ from relational databases in how they organize data as well as in the query languages they support. Document databases are usually categorized under the NoSQL umbrella term.

Collection and Documents

- The Folder & File Analogy 📁
- Think of MongoDB like a file cabinet on your computer:

```
MongoDB Database (The Whole Cabinet)  
├──  
│   └── Collection (A Folder)  
│       └── Document (A Single JSON File inside the folder)
```

1. What is a Document? (A Single Item)

- A Document is just ONE individual record stored in JSON (BSON) format.

```
{  
  "_id": "64f1a2b3c4d5e6f7a8b9c0d1",
```

```
"name": "Arto Hellas",  
"number": "040-123456"  
}
```

2. What is a Collection? (The Group / Array)

- A Collection is simply a group or list of documents (equivalent to an array of objects in JavaScript).
- Example:

```
const persons = [  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456}  
]
```

- The whole **array persons** is called the **Collection**.
- ****Each individual object ****{ id: "1", ... } inside that array is **called a Document**.

Process of db connection

- create project first ==> cluster(db) ==> set_user (name,password,role) => IP address list allow for all by (0.0.0.0/0) else only specific for you by default already added you IP add.
- **install mongoose** into backend folder
- mkdir file called mongo.js init all requirements into this file.
- Connection code
- code

```
const mongoose = require('mongoose');  
  
// 1. Check if the user passed a password in the terminal command  
if (process.argv.length < 3) {  
  console.log('give password as argument');  
  process.exit(1);  
}  
  
// 2. Read the password from the 3rd command-line argument (process.argv[2])  
// node mongo myPassword123 [3]  
const password = process.argv[2];  
  
// 3. The MongoDB Connection String (Points to your Atlas cloud database)  
const url = `mongodb+srv://fso-vinayak:${password}@notes-  
app.loz4osq.mongodb.net/notes-app?appName=notes-app`
```

```
mongoose.set('strictQuery', false);

// 4. Open the connection to the cloud database
console.log('Connecting to MongoDB Atlas...');
mongoose.connect(url);

// Define the Schema (The Blueprint of what a Note looks like)
const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean
});

// Model Constructor Capitalization: Since mongoose.model(...) returns a
// Constructor Function (a factory to create new objects), convention dictates
// using PascalCase:

// 6. Create the Model (The Constructor that talks to the 'notes' collection)

const Note = mongoose.model('Note', noteSchema);

// If you ever desperately wanted it to be named persons instead of
// people, Mongoose lets you pass a custom collection name as the 3rd argument:
// mongoose.model('Person', personSchema, 'persons'),
// but letting Mongoose do its default people is standard practice

// 7. Create a new Note document object
const note = new Note({
  content: 'HTML is easy and fun',
  important: true
});

// 8. Save the note to the cloud database and CLOSE the connection
note.save().then(result => {
  console.log('note saved');
  mongoose.connection.close(); // 👉 Must close the connection so the
  // script exits!
});
```

◦ Explanation :-

```
const mongoose = require('mongoose');
```

1. The Command-Line Argument: **process.argv**

```
if(process.argv.length < 3){
  console.log('give password as argument');
```

```
    process.exit(1);
  }
  <!-- Read the password from the 3rd command-line argument (process.argv[2]) -
  ->
  const password = process.argv[2];
```

- **When you type in terminal: node mongo.js myPassword123**
- Node creates an array called process.argv:
 - [node ,mongo.js ,myPassword123]
 - 1. process.argv[0] = Path to Node engine (node)
 - 2. process.argv[1] = Path to the file (mongo.js)
 - 3. process.argv[2] = Whatever you typed 3rd: 'myPassword123'!
- If you forgot to type the password, process.argv.length is less than 3, so process.exit(1) immediately stops the script with an error code!

2. The Connection URL & mongoose.connect(url)

```
<!-- The MongoDB Connection String (Points to your Atlas cloud database) -->

const url = `mongodb+srv://fso-vinayak:${password}@notes-
app.loz4osq.mongodb.net/notes-app?appName=notes-app`

mongoose.set('strictQuery', false);
mongoose.connect(url)
mongoose.connect(url,{family:4})
```

- **url**: Embeds your dynamic password and tells Mongoose to connect to the noteApp database.
- **strictQuery: true**: Mongoose strictly ignores any search field that is not written in your Schema.
- **strictQuery: false**: Mongoose allows flexible searching in the database and silences annoying terminal warning messages.
- **mongoose.connect(url)**: Opens the network TCP socket connection across the internet to the MongoDB Atlas servers.
- Passing { family: 4 } tells Mongoose:
 - "Hey Mongoose, force the connection to strictly use IPv4 so my connection never hangs or lags!"
 - If your mongo.js connected in 1 second without errors (which it did on your screenshot!), your network is already working fine!.

- But if you ever run into a weird connection timeout error on a different Wi-Fi, adding { family: 4 } is a handy networking fix.

The Schema (The Architectural Blueprint)

```
const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean,
});
```

- MongoDB by itself is **"schemaless"** (*it blindly accepts any garbage JSON*).
- Mongoose Schemas give your app strict rules:
 - Every note MUST have content as a String and important as a Boolean.

4. The Model (The Constructor) & Mongoose Naming Magic

```
const Note = mongoose.model('Note', noteSchema);
```

- Note is now a JavaScript Constructor function (like a Class) that lets us create, find, update, and delete notes.
- **Mongoose's Secret Pluralization Magic** : When you name your model singular capitalized 'Note', Mongoose automatically lowercases and pluralizes it to name the collection in the database notes! (e.g., 'Person' becomes 'people' or 'persons', 'User' becomes 'users').

Creating & Saving the Document (Async Promise!)

```
const note = new Note({
  content: 'HTML is easy and fun',
  important: true,
});

note.save().then(result => {
  console.log('note saved!');
});

mongoose.connection.close(); // 🙋 WHY IS THIS HERE?
});
```

- new Note({ ... }) creates the document object in RAM.
- note.save() sends the insert command over the internet to MongoDB Atlas on disk.

- **note.save() returns a PROMISE!** When MongoDB confirms the data is saved, the `.then()` callback runs!

-
- ⚠ Why `mongoose.connection.close()` is Mandatory in CLI scripts:
 - Node.js keeps the database network connection open forever by default.
 - If you do NOT call `mongoose.connection.close()`, your script will finish saving, but your terminal will hang and freeze forever, waiting for more commands!
 - Calling `mongoose.connection.close()` cuts the network wire so your terminal returns cleanly to the command line prompt!
-

Now: How to FETCH / READ data from MongoDB (`Note.find()`);

- Now let's see how Mongoose reads all notes from the database.
- In `mongo.js`, you can replace the save code with **`Note.find({})`**:
 - What `{}` means in **`Note.find({})`**:
 - **The empty object `{}` is a filter search query that means "Find EVERYTHING without restrictions!"**
 - **If you wanted only important notes, you would write: `Note.find({ important: true })`.**

```
Note.find({}).then(result =>{
  result.forEach(note =>{
    console.log(note);
  });
  mongoose.connection.close().
});
`
```

Connection the backend to a Database

```
const mongoose = require('mongoose');

const url = process.env.MONGODB_URI
mongoose.set('strictQuery', false);

console.log('Connecting to MongoDB Atlas...');

mongoose.connect(url)
  .then(() => {
    console.log('connected to MongoDB');
  })
```

```
.catch(error => {
  console.log('error connecting to MongoDB:', error.message);
})

const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean
});

noteSchema.set('toJSON', {
  transform: (document, returnedObject) => {
    returnedObject.id = returnedObject._id.toString()
    delete returnedObject._id
    delete returnedObject.__v
  }
});

const Note = mongoose.model('Note', noteSchema);

module.exports = Note;
```

URI Vs URL

- URI = Uniform Resource Identifier (The general identity)
- URL = Uniform Resource Locator (The specific location / address)
- **The Rule: Every URL is a URI, but not every URI is a URL.**